Neural Network

Contents

- Neural Network
- Neuron
- Network Architecture
- Backpropagation
- Bayesian belief Network

Neural Networks

- Artificial neural network (ANN) is a machine learning approach that models human brain and consists of a number of artificial neurons.
- Neuron in ANNs tend to have fewer connections than biological neurons.
- Each neuron in ANN receives a number of inputs.
- An activation function is applied to these inputs which results in activation level of neuron (output value of the neuron).
- Knowledge about the learning task is given in the form of examples called training examples.

Neural Network (Cont….)

- An Artificial Neural Network is specified by:
  - **neuron model**: the information processing unit of the NN,
  - **an architecture**: a set of neurons and links connecting neurons. Each link has a weight,
  - **a learning algorithm**: used for training the NN by modifying the weights in order to model a particular learning task correctly on the training examples.
- The aim is to obtain a NN that is trained and generalizes well.
- It should behave correctly on new instances of the learning task.

Neuron

- The neuron is the basic information processing unit of a NN. It consists of:
  - A set of links, describing the neuron inputs, with weights W_1, W_2, ..., W_m
  - An adder function (linear combiner) for computing the weighted sum of the inputs:

    (real numbers)

$$\mathbf{netj} = \sum_{j=1}^{m} \mathbf{w_j x_j}$$

  - Activation function φ is applied to the net input.
  - Bias act as a threshold.

Neuron diagram

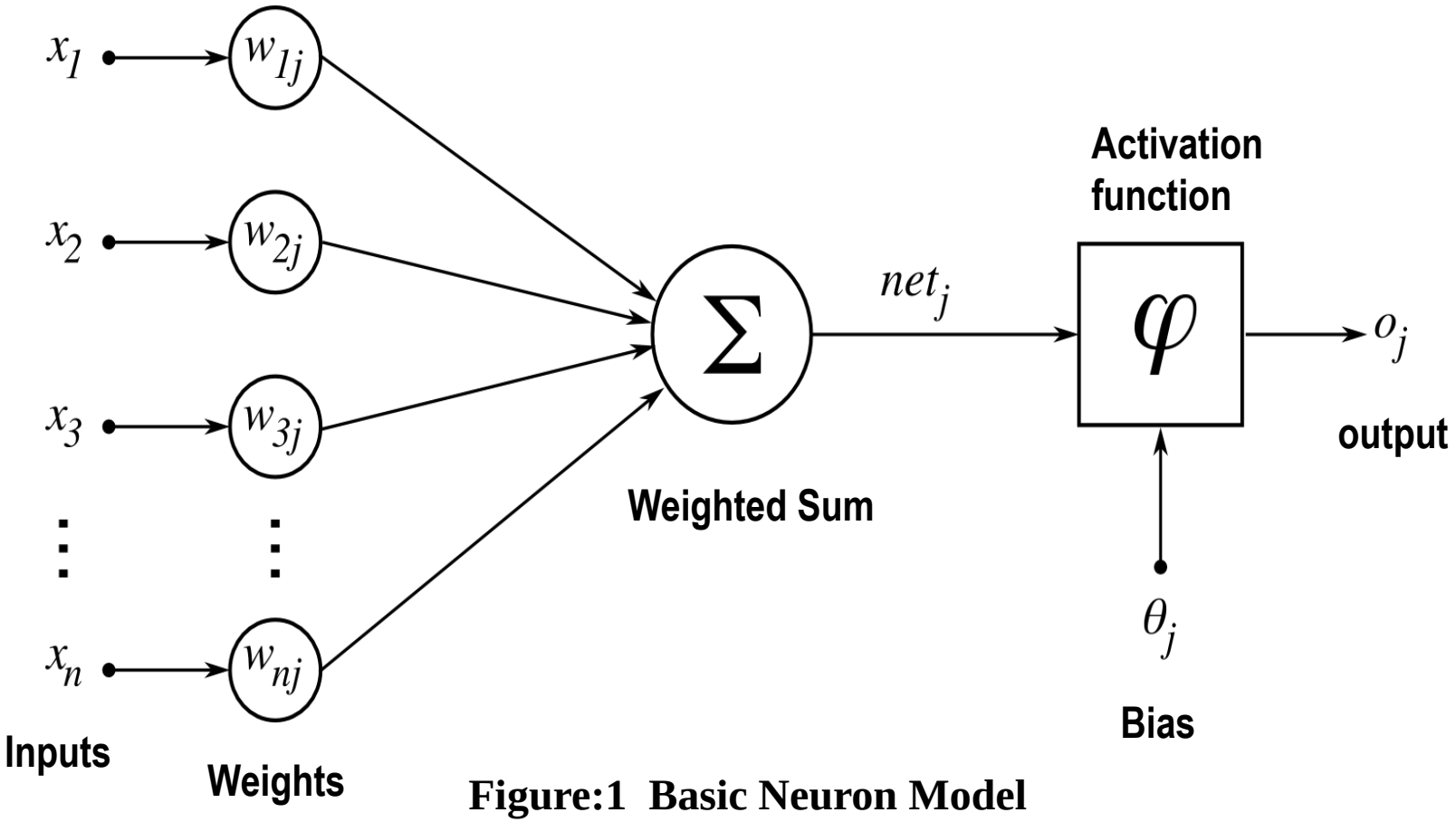


Figure:1 Basic Neuron Model

Neuron Models

- The choice of activation function φ determines the neuron model.

Examples:

- step function: $\varphi(x) = \begin{cases} 0 & \text{if } x < \theta \\ 1 & \text{if } x > \theta \end{cases}$

- sigmoid function $\varphi(x) = \dfrac{1}{1+\exp(-x)}$

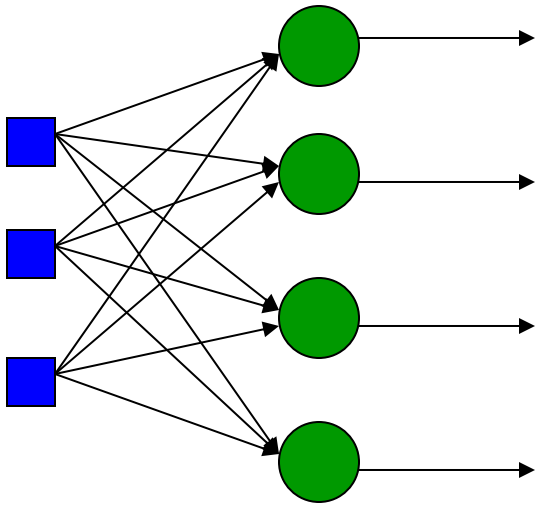
- Gaussian function: $\varphi(x) = \dfrac{1}{\sqrt{2\pi}\sigma} \exp\left(-\dfrac{1}{2}\left(\dfrac{x-\mu}{\sigma} \right)^2 \right)$

Network Architectures

- Different classes of network architectures
  - single-layer feed-forward
  - multi-layer feed-forward

Single Layer Feed-forward

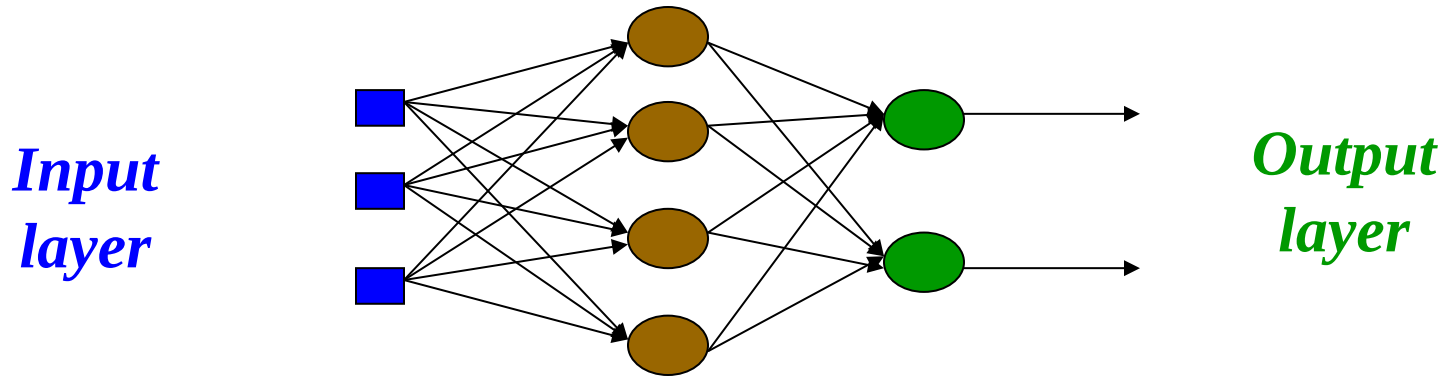
Input layer of source nodes



Output layer of neurons

Multi layer feed-forward NN (FFNN)

- FFNN is a more general network architecture, where there are hidden layers between input and output layers.
- Hidden nodes do not directly receive inputs nor send outputs to the external environment.
- FFNNs overcome the limitation of single-layer NN.
- They can handle non-linearly separable learning tasks.



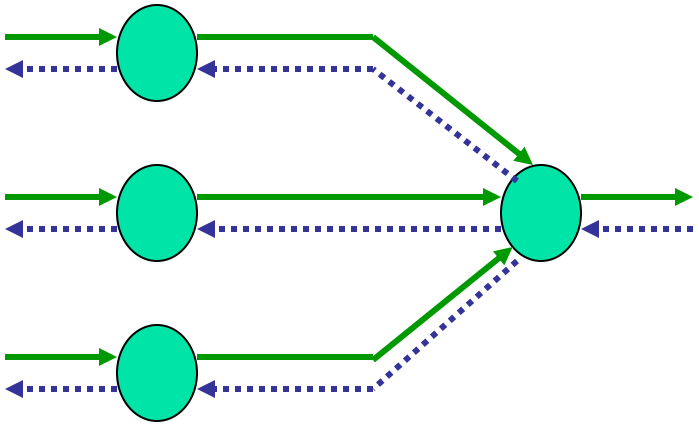
3-4-2 Network

Training Algorithm: Backpropagation

- The Backpropagation algorithm learns in the same way as single perceptron.
- It searches for weight values that minimize the total error of the network over the set of training examples (training set).
- Backpropagation consists of the repeated application of the following two passes:
  - **Forward pass**: In this step, the network is activated on one example and the error of (each neuron of) the output layer is computed.
  - **Backward pass**: in this step the network error is used for updating the weights. The error is propagated backwards from the output layer through the network layer by layer.

Backpropagation

- Back-propagation training algorithm



Network activation
Forward Step

Error propagation
Backward Step

- Backpropagation adjusts the weights of the NN in order to minimize the network total mean squared error.

Backpropagation Algorithm

- Neural network learning for classification or numeric prediction, using the backpropagation algorithm.
- **Input:**

  D, a data set consisting of the training examples and their associated target values;

  l, the learning rate;

  network, a multilayer feed-forward network.
- **Output:** A trained neural network.

Backpropagation Algorithm (Cont…)

(1) Initialize all weights and biases in *network*;
(2) **while** terminating condition is not satisfied {
(3) **for** each training tuple $\mathbf{X}$ in D {
(4) // Propagate the inputs forward:
(5) **for** each input layer unit j {
(6) $O_j = I_j$; // output of an input unit is its actual input value
(7) **for** each hidden or output layer unit j {
(8) $I_j = \sum_i w_{ij} O_i + \theta_j$; //compute the net input of unit j with respect to the previous layer, i
(9) $O_j = \frac{1}{1+e^{-I_j}}$; } // compute the output of each unit j
(10) // Backpropagate the errors:
(11) **for** each unit j in the output layer
(12) $Err_j = O_j(1 - O_j)(T_j - O_j)$; // compute the error
(13) **for** each unit j in the hidden layers, from the last to the first hidden layer
(14) $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$; // compute the error with respect to the next higher layer, k
(15) **for** each weight w_{ij} in *network* {
(16) $\Delta w_{ij} = (l) Err_j O_i$; // weight increment
(17) $w_{ij} = w_{ij} + \Delta w_{ij}$; } // weight update
(18) **for** each bias θ_j in *network* {
(19) $\Delta \theta_j = (l) Err_j$; // bias increment
(20) $\theta_j = \theta_j + \Delta \theta_j$; } // bias update
(21) } }



Backpropagation Algorithm (Cont…)

- **Initialize the weights:**

The weights in the network are initialized to small random numbers

(e.g., ranging from -1.0 to 1.0, or -0.5 to 0.5). Each unit has a *bias* associated with it, the biases are similarly initialized to small random numbers.

Each training example, ***X,*** is processed by the following steps:

- **Propagate the inputs forward:**

- Consider a network of three layers.
- Let us use i to represent nodes in input layer, j to represent nodes in hidden layer and output layer.
- w_{ij} refers to weight of connection from unit ***i*** in the previous layer to unit ***j.***

Backpropagation Algorithm (Cont…)

- Given a unit, ***j*** in a hidden or output layer, the net input, I_j , to unit ***j*** is

$$I_j = \sum_i w_{ij} O_i + \theta_j$$

Where $\mathbf{O}_i$ is the output of unit i and θ_j is threshold for unit j.

- Given the net input I_j to unit j, then O_j , the output of unit j, is computed as

$$O_j = \frac{1}{1 + e^{-I_j}}$$

Backpropagation Algorithm (Cont…)

- **Backpropagate the error:**

The error is propagated backward by updating the weights and biases to reflect the error of the network's prediction.

For a unit *j in the output* layer, the error Err_j *is computed by*

$$Err_j = O_j(1 - O_j)(T_j - O_j)$$

where O_j is the actual output of unit j, and T_j is the known target value of the given training example.

The error of a hidden layer unit *j is*

$$Err_j = O_j(1 - O_j)\sum_k Err_k w_{jk}$$

where w_{jk} is the weight of the connection from unit j to a unit k in the next higher layer, and $\boldsymbol{Err_k}$ is the error of unit k.

Backpropagation Algorithm (Cont…)

- The weights and biases are updated to reflect the propagat e errors. Weights are updated by the following equations, where Δwij is the change in weight wij :

$$\Delta w_{ij} = (l) Err_j O_i$$

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

The variable *l is the* ***learning rate, a constant typically having*** a value between 0.0 and 1.0.

Biases are updated by the following equations, where $\Delta\theta_j$ is the change in bias θ_j :

$$\Delta \theta_j = (l) Err_j.$$

$$\theta_j = \theta_j + \Delta \theta_j.$$

Stopping criterions

- Total mean squared error change:
  - Back-prop is considered to have converged when the absolute rate of change in the average squared error per epoch is sufficiently small

    (in the range [0.1, 0.01])

- All Δw_{ij} in the previous epoch are so small as to be below some specified threshold

  or

- A prespecified number of epochs has expired.

Example

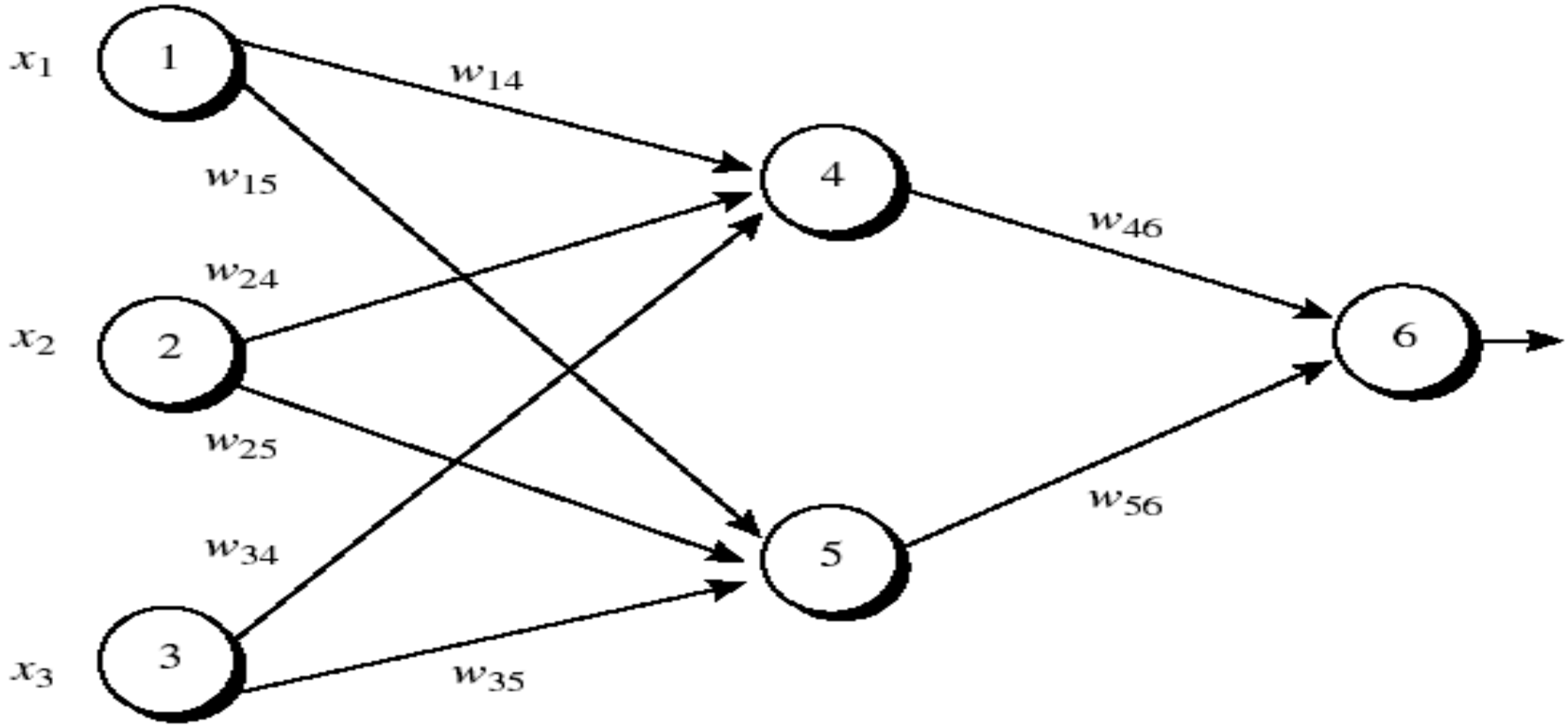


Figure 1: An example of a multilayer feed-forward neural network. Assume that the learning rate ***l*** is 0.9 and the first training example, X = (1,0,1) whose class label is 1.

Note: The sigmoid function is applied to hidden layer and output layer.

Example

***Table 1*: Initial input and weight values**

x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

***Table 2*: The net input and output calculation**

Unit j	Net input I_j	Output O_j
4	0.2 + 0 -0.5 -0.4 = -0.7	$1/(1+e^{0.7})=0.332$
5	-0.3 +0+0.2 +0.2 =0.1	$1/(1+e^{0.1})=0.525$
6	(-0.3)(0.332)-(0.2)(0.525)+0.1 = -0.105	$1/(1+e^{0.105})=0.474$

***Table 3:* Calculation of the error at each node**

Unit j	Err_j
6	(0.474)(1-0.474)(1-0.474)=0.1311
5	(0.525)(1-0.525)(0.1311)(-0.2)=-0.0065
4	(0.332)(1-0.332)(0.1311)(-0.3)=-0.0087

Example

***Table 4*: Calculation for weight and Bias updating**

Weight	New value
w_{46}	-03+(0.9)(0.1311)(0.332)= -0.261
w_{56}	-0.2+(0.9)(0.1311)(0.525)= -0.138
w_{14}	0.2 +(0.9)(-0.0087)(1) = 0.192
w_{15}	-0.3 +(0.9)(-0.0065)(1) = -0.306
w_{24}	0.4+ (0.9)(-0.0087)(0) = 0.4
w_{25}	0.1+ (0.9)(-0.0065)(0) = 0.1
w_{34}	-0.5+ (0.9)(-0.0087)(1) = -0.508
w_{35}	0.2 + (0.9)(-0.0065)(1) = 0.194
θ_6	0.1 + (0.9)(0.1311) = 0.218
θ_5	0.2 + (0.9)(-0.0065)=0.194
θ_4	-0.4 +(0.9)(-0.0087) = -0.408

Example

- The old error was 0.526 [($Target_j$ - $Output_j$)= 1- 0.474]

- After adjusting weight, the new error is 0.515.

Therefore, error has reduced.

ANN Example

Consider a simple Artificial Neural Network (ANN) with the following structure:

- **Input Layer**: 3 inputs with values $x_1 = 1$, $x_2 = 0$, and $x_3 = 1$.
- **Hidden Layer**: 2 nodes, each with a randomly assigned weight. The network has the following weights and biases:
  - Weights between input and hidden layer:
    - $w_{11} = 0.2$, $w_{12} = -0.1$, $w_{13} = 0.5$ (for the first hidden node)
    - $w_{21} = 0.7$, $w_{22} = -0.4$, $w_{23} = 0.2$ (for the second hidden node)
  - Bias for the first hidden node $b_1 = 0.1$
  - Bias for the second hidden node $b_2 = -0.3$

ANN Example

- **Output Layer**: 1 node with the following weights and bias:
  - Weights between hidden and output layer:
    - $w_{o1} = 0.6$ (for the first hidden node)
    - $w_{o2} = -0.2$ (for the second hidden node)
  - Bias for the output node $b_o = 0.05$

The activation function for the hidden and output layers is the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

ANN Example

Given the input $[1, 0, 1]$ and the target output $y = 1$, calculate the output of the network during the first iteration.

1. Compute the weighted sums for the hidden layer nodes.
2. Apply the sigmoid activation to these sums.
3. Compute the weighted sum for the output node.
4. Apply the sigmoid activation to the output node to obtain the final output.

ANN Example

You can follow these steps to perform the first iteration manually:

- **Step 1**: Calculate the weighted sum z_1 and z_2 for the two hidden layer nodes.

$$z_1 = w_{11} \cdot x_1 + w_{12} \cdot x_2 + w_{13} \cdot x_3 + b_1$$

$$z_2 = w_{21} \cdot x_1 + w_{22} \cdot x_2 + w_{23} \cdot x_3 + b_2$$

- **Step 2**: Apply the sigmoid activation function to z_1 and z_2.

$$h_1 = \sigma(z_1), \quad h_2 = \sigma(z_2)$$

- **Step 3**: Calculate the weighted sum for the output layer node.

$$z_o = w_{o1} \cdot h_1 + w_{o2} \cdot h_2 + b_o$$

- **Step 4**: Apply the sigmoid activation function to z_o to get the final output:

$$\hat{y} = \sigma(z_o)$$

Example

Step 1: Calculate the weighted sums for the hidden layer nodes

Given inputs:

$$x_1 = 1, \quad x_2 = 0, \quad x_3 = 1$$

Weights between input and hidden layer:

- For the first hidden node:

$$w_{11} = 0.2, \quad w_{12} = -0.1, \quad w_{13} = 0.5, \quad b_1 = 0.1$$

- For the second hidden node:

$$w_{21} = 0.7, \quad w_{22} = -0.4, \quad w_{23} = 0.2, \quad b_2 = -0.3$$

ANN Example

For the first hidden node (z_1):

$$z_1 = w_{11} \cdot x_1 + w_{12} \cdot x_2 + w_{13} \cdot x_3 + b_1$$

$$z_1 = (0.2 \cdot 1) + (-0.1 \cdot 0) + (0.5 \cdot 1) + 0.1$$

$$z_1 = 0.2 + 0 + 0.5 + 0.1 = 0.8$$

For the second hidden node (z_2):

$$z_2 = w_{21} \cdot x_1 + w_{22} \cdot x_2 + w_{23} \cdot x_3 + b_2$$

$$z_2 = (0.7 \cdot 1) + (-0.4 \cdot 0) + (0.2 \cdot 1) + (-0.3)$$

$$z_2 = 0.7 + 0 + 0.2 - 0.3 = 0.6$$

ANN Example

Step 2: Apply the sigmoid activation function to these sums

The sigmoid function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

For the first hidden node (h_1):

$$h_1 = \sigma(z_1) = \frac{1}{1 + e^{-0.8}}$$

First, calculate $e^{-0.8}$:

$$e^{-0.8} \approx 0.449$$

Now, compute h_1:

$$h_1 = \frac{1}{1 + 0.449} = \frac{1}{1.449} \approx 0.690$$

ANN Example

For the second hidden node (h_2):

$$h_2 = \sigma(z_2) = \frac{1}{1 + e^{-0.6}}$$

First, calculate $e^{-0.6}$:

$$e^{-0.6} \approx 0.549$$

Now, compute h_2:

$$h_2 = \frac{1}{1 + 0.549} = \frac{1}{1.549} \approx 0.645$$

ANN Example

Step 3: Calculate the weighted sum for the output layer node

Weights between hidden and output layer:

$$w_{o1} = 0.6, \quad w_{o2} = -0.2, \quad b_o = 0.05$$

Now, calculate the weighted sum z_o for the output node:

$$z_o = w_{o1} \cdot h_1 + w_{o2} \cdot h_2 + b_o$$

$$z_o = (0.6 \cdot 0.690) + (-0.2 \cdot 0.645) + 0.05$$

$$z_o = 0.414 - 0.129 + 0.05 = 0.335$$

ANN Example

Step 4: Apply the sigmoid activation function to the output

Now, apply the sigmoid activation function to z_o:

$$\hat{y} = \sigma(z_o) = \frac{1}{1 + e^{-0.335}}$$

First, calculate $e^{-0.335}$:

$$e^{-0.335} \approx 0.715$$

Now, compute $\hat{y}$:

$$\hat{y} = \frac{1}{1 + 0.715} = \frac{1}{1.715} \approx 0.583$$

Example

Final Output

The output of the network for the first iteration is approximately:

$$\hat{y} \approx 0.583$$

This is the predicted output for the given input.

Batch Normalization Example

To apply **Batch Normalization** to the neural network problem, we will modify the network by normalizing the activations of the hidden layer before applying the activation function (sigmoid in this case). Batch normalization is typically applied to each layer before the activation function to stabilize learning.

Batch Normalization Example

Batch Normalization Formula

Given a batch of pre-activation values (we have only two nodes in the hidden layer), we normalize each pre-activation z using the following formulas:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} z_i$$

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(z_i - \mu_B)^2$$

$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Where μ_B is the mean of the batch, σ_B^2 is the variance of the batch, $\hat{z}_i$ is the normalized value, and ϵ is a small number added for numerical stability (typically $\epsilon = 10^{-5}$).

After normalization, we apply a learnable scaling factor γ and a shift factor β:

$$z_i^{BN} = \gamma \hat{z}_i + \beta$$

Finally, we pass the normalized and scaled values through the activation function.

Batch Normalization Example

Step-by-Step Calculation with Batch Normalization

1. Compute the Pre-Activation Values (as done previously)

From the previous step, we already calculated the pre-activation values z_1 and z_2 for the hidden layer nodes:

$$z_1 = 0.8, \quad z_2 = 0.6$$

These values represent the input to the batch normalization layer.

Batch Normalization Example

2. Compute the Mean and Variance for Batch Normalization

Since we have only two hidden nodes in this example, our "batch" consists of these two pre-activation values, z_1 and z_2. So we calculate the mean μ_B and variance σ_B^2 for these values.

Mean:

$$\mu_B = \frac{1}{2} \cdot (z_1 + z_2) = \frac{1}{2} \cdot (0.8 + 0.6) = 0.7$$

Variance:

$$\sigma_B^2 = \frac{1}{2} \cdot \left((z_1 - \mu_B)^2 + (z_2 - \mu_B)^2\right)$$

$$\sigma_B^2 = \frac{1}{2} \cdot \left((0.8 - 0.7)^2 + (0.6 - 0.7)^2\right)$$

$$\sigma_B^2 = \frac{1}{2} \cdot (0.01 + 0.01) = 0.01$$

Batch Normalization Example

3. Normalize the Pre-Activation Values

Now, we normalize the pre-activation values z_1 and z_2 using the calculated mean and variance:

For z_1:

$$\hat{z}_1 = \frac{z_1 - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} = \frac{0.8 - 0.7}{\sqrt{0.01 + 10^{-5}}} \approx \frac{0.1}{0.10005} \approx 0.9995$$

For z_2:

$$\hat{z}_2 = \frac{z_2 - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} = \frac{0.6 - 0.7}{\sqrt{0.01 + 10^{-5}}} \approx \frac{-0.1}{0.10005} \approx -0.9995$$

Batch Normalization Example

4. Apply Scaling (γ) and Shifting (β)

For simplicity, assume the learnable parameters $\gamma = 1$ and $\beta = 0$ (common initial values for batch normalization). Thus, the normalized values remain the same as the output of this step:

$$z_1^{BN} = \gamma \hat{z}_1 + \beta = 1 \cdot 0.9995 + 0 = 0.9995$$

$$z_2^{BN} = \gamma \hat{z}_2 + \beta = 1 \cdot (-0.9995) + 0 = -0.9995$$

Batch Normalization Example

5. Apply the Sigmoid Activation Function

Next, we apply the sigmoid activation function to the batch-normalized values z_1^{BN} and z_2^{BN}.

For the first hidden node:

$$h_1^{BN} = \sigma(z_1^{BN}) = \frac{1}{1 + e^{-0.9995}} \approx \frac{1}{1 + 0.368} \approx 0.731$$

For the second hidden node:

$$h_2^{BN} = \sigma(z_2^{BN}) = \frac{1}{1 + e^{0.9995}} \approx \frac{1}{1 + 2.717} \approx 0.269$$

Batch Normalization Example

6. Compute the Output Layer Pre-Activation Value

Now that we have the batch-normalized activations of the hidden layer, we can proceed as before to compute the pre-activation value of the output layer:

$$z_o = w_{o1} \cdot h_1^{BN} + w_{o2} \cdot h_2^{BN} + b_o$$

$$z_o = (0.6 \cdot 0.731) + (-0.2 \cdot 0.269) + 0.05$$

$$z_o = 0.439 - 0.054 + 0.05 = 0.435$$

Batch Normalization Example

7. Apply the Sigmoid Activation Function to the Output

Finally, we apply the sigmoid function to the output pre-activation value:

$$\hat{y} = \sigma(z_o) = \frac{1}{1 + e^{-0.435}}$$

First, calculate $e^{-0.435}$:

$$e^{-0.435} \approx 0.647$$

Now, compute $\hat{y}$:

$$\hat{y} = \frac{1}{1 + 0.647} = \frac{1}{1.647} \approx 0.607$$

Batch Normalization Example

Final Output with Batch Normalization

The output of the network after applying batch normalization is approximately:

$$\hat{y} \approx 0.607$$

This output is slightly different from the previous result (0.583), showing how batch normalization affects the training by stabilizing and normalizing the activations.

Dropout Example

Dropout is a regularization technique that helps prevent overfitting in neural networks by randomly "dropping out" or ignoring a fraction of neurons during the training process. This forces the network to learn redundant representations, as no single neuron can rely solely on others. The process of dropping neurons helps in generalizing the model to new data and improving performance.

How Dropout Works:

1. During each training step, dropout randomly sets the output of a fraction of the neurons (typically 20%-50%) to zero.
2. The neurons that are "dropped" are ignored for both forward and backward propagation.
3. During inference (testing), all neurons are used, but the outputs are scaled by the probability p (the dropout probability). This scaling ensures the expected activation values remain the same.

Dropout Example

Example:

Let's incorporate dropout into the **previous example** (ANN with 3 inputs, 2 hidden nodes, and 1 output node). We'll assume a dropout rate of 50% (i.e., each hidden node has a 50% chance of being dropped during training).

1. **Input layer**: 3 input nodes with values $[1, 0, 1]$.
2. **Hidden layer**: 2 hidden nodes with dropout. Each hidden node has a 50% chance of being dropped during training.
3. **Output layer**: 1 output node for binary classification.

Dropout Example

Dropout Procedure in Steps

Step 1: Initial Calculations Without Dropout

Recall the previous steps for calculating the output without dropout:

- Pre-activation values for hidden nodes:
  - $z_1 = 0.8$
  - $z_2 = 0.6$
- Sigmoid activations:
  - $h_1 = \sigma(0.8) \approx 0.690$
  - $h_2 = \sigma(0.6) \approx 0.645$

Dropout Example

Step 2: Apply Dropout to Hidden Nodes

Since we are applying **50% dropout**, each hidden node has a 50% probability of being dropped. For this particular training iteration, let's randomly drop **one hidden node** (this randomness happens during each iteration).

- Assume that h_2 **is dropped** during this iteration.
- The remaining node, h_1, stays active, and its value is **not scaled** during training.

Step 3: Update Activations After Dropout

After applying dropout, we have the following:

- h_1 stays: $h_1 \approx 0.690$
- h_2 is dropped: $h_2 = 0$

Dropout Example

Step 4: Compute Output Layer Pre-Activation

Using only the active hidden node h_1, we calculate the output node's pre-activation value z_o:

$$z_o = w_{o1} \cdot h_1 + w_{o2} \cdot h_2 + b_o$$

Since $h_2 = 0$ (due to dropout):

$$z_o = (0.6 \cdot 0.690) + (-0.2 \cdot 0) + 0.05$$

$$z_o = 0.414 + 0 + 0.05 = 0.464$$

Dropout Example

Step 5: Apply Sigmoid Activation to Output

Now, apply the sigmoid activation function to the output node:

$$\hat{y} = \sigma(0.464) = \frac{1}{1 + e^{-0.464}} \approx \frac{1}{1 + 0.629} \approx 0.614$$

So, the output for this iteration (with dropout) is $\hat{y} \approx 0.614$.

Batch Normalization with Dropout Example

Let's now combine **Batch Normalization** and **Dropout** in the same neural network to see how these techniques work together.

Setup:

1. **Batch Normalization:** Applied before the activation function (sigmoid) to standardize the pre-activation values.
2. **Dropout:** Applied after the batch normalization and activation function, randomly dropping neurons to prevent overfitting.

We will follow the same neural network architecture:

- **Input layer:** 3 input nodes with values $[1, 0, 1]$
- **Hidden layer:** 2 hidden nodes with batch normalization and dropout (50% dropout rate)
- **Output layer:** 1 output node

Batch Normalization with Dropout Example

Steps for Calculations:

Step 1: Pre-Activation Values Without Dropout and Batch Normalization

We already calculated the pre-activation values of the hidden nodes in previous steps:

- $z_1 = 0.8$
- $z_2 = 0.6$

These are the raw pre-activation values before applying batch normalization.

Batch Normalization with Dropout Example

Step 2: Apply Batch Normalization to Hidden Layer

Batch normalization is applied before the activation function. Using the same mean and variance calculated previously:

- Mean $\mu_B = 0.7$
- Variance $\sigma_B^2 = 0.01$

We now normalize the pre-activation values.

For z_1:

$$\hat{z}_1 = \frac{z_1 - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} = \frac{0.8 - 0.7}{\sqrt{0.01 + 10^{-5}}} \approx 0.9995$$

For z_2:

$$\hat{z}_2 = \frac{z_2 - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} = \frac{0.6 - 0.7}{\sqrt{0.01 + 10^{-5}}} \approx -0.9995$$

Batch Normalization with Dropout Example

Step 3: Apply Activation Function (Sigmoid)

Now, we apply the sigmoid activation function to the normalized pre-activation values:

For z_1:

$$h_1 = \sigma(0.9995) = \frac{1}{1 + e^{-0.9995}} \approx 0.731$$

For z_2:

$$h_2 = \sigma(-0.9995) = \frac{1}{1 + e^{0.9995}} \approx 0.269$$

So, the activations of the hidden layer nodes before dropout are:

- $h_1 \approx 0.731$
- $h_2 \approx 0.269$

Batch Normalization with Dropout Example

Step 4: Apply Dropout to Hidden Layer

With **50% dropout**, each hidden node has a 50% chance of being dropped during this training iteration. For this example, we assume h_2 **is dropped** during this iteration, and only h_1 is kept active.

- $h_1 = 0.731$
- $h_2 = 0$ (dropped)

Step 5: Compute Pre-Activation for the Output Node

Now, we use the active hidden node h_1 to compute the pre-activation value of the output node z_o:

$$z_o = w_{o1} \cdot h_1 + w_{o2} \cdot h_2 + b_o$$

Since $h_2 = 0$:

$$z_o = (0.6 \cdot 0.731) + (-0.2 \cdot 0) + 0.05 = 0.4386 + 0 + 0.05 = 0.4886$$

Batch Normalization with Dropout Example

Step 6: Apply Sigmoid Activation to Output Node

Now, apply the sigmoid function to the output node:

$$\hat{y} = \sigma(0.4886) = \frac{1}{1 + e^{-0.4886}} \approx 0.620$$

So, the output for this training iteration (with both **Batch Normalization** and **Dropout**) is:

$$\hat{y} \approx 0.620$$

Inference Mode (No Dropout, With Batch Normalization)

How Dropout Works

1. **Training Phase:**
   - During training, a random subset of neurons is deactivated with a certain probability (dropout rate).
   - The network trains on this reduced subset.
2. **Inference Phase:**
   - At inference time, all neurons are used, but their activations are scaled down by the dropout rate to account for the fact that they were not always active during training.

Inference Mode (No Dropout, With Batch Normalization)

Why Does Dropout Help?

By randomly dropping neurons, the network is **forced to learn without relying too heavily on specific neurons** like h_2. In another training step, maybe h_1 will be dropped, and the network will have to rely on h_2 instead. This forces the network to distribute learning across different neurons, instead of depending on just one or two neurons.

This "redundancy" (where the network learns the same information in different ways) makes the network more flexible and robust. Even if some neurons are ignored during training, the network can still make good predictions.

Inference Mode (No Dropout, With Batch Normalization)

During inference, dropout is not applied, and all hidden nodes are used. However, the outputs of the hidden nodes are scaled by the dropout probability p (in this case, $p = 0.5$).

Step 1: Scale Hidden Node Activations After Batch Normalization

During inference, each hidden node's activation is multiplied by 0.5 to account for the dropout used during training.

- $h_1 = 0.731 \times 0.5 = 0.3655$
- $h_2 = 0.269 \times 0.5 = 0.1345$

Inference Mode (No Dropout, With Batch Normalization)

Step 2: Compute Pre-Activation for Output Node

Using the scaled hidden node activations:

$$z_o = (0.6 \cdot 0.3655) + (-0.2 \cdot 0.1345) + 0.05$$

$$z_o = 0.2193 - 0.0269 + 0.05 = 0.2424$$

Step 3: Apply Sigmoid Activation to Output Node

Now, apply the sigmoid function to the output node:

$$\hat{y} = \sigma(0.2424) = \frac{1}{1 + e^{-0.2424}} \approx 0.560$$

Inference Mode (No Dropout, With Batch Normalization)

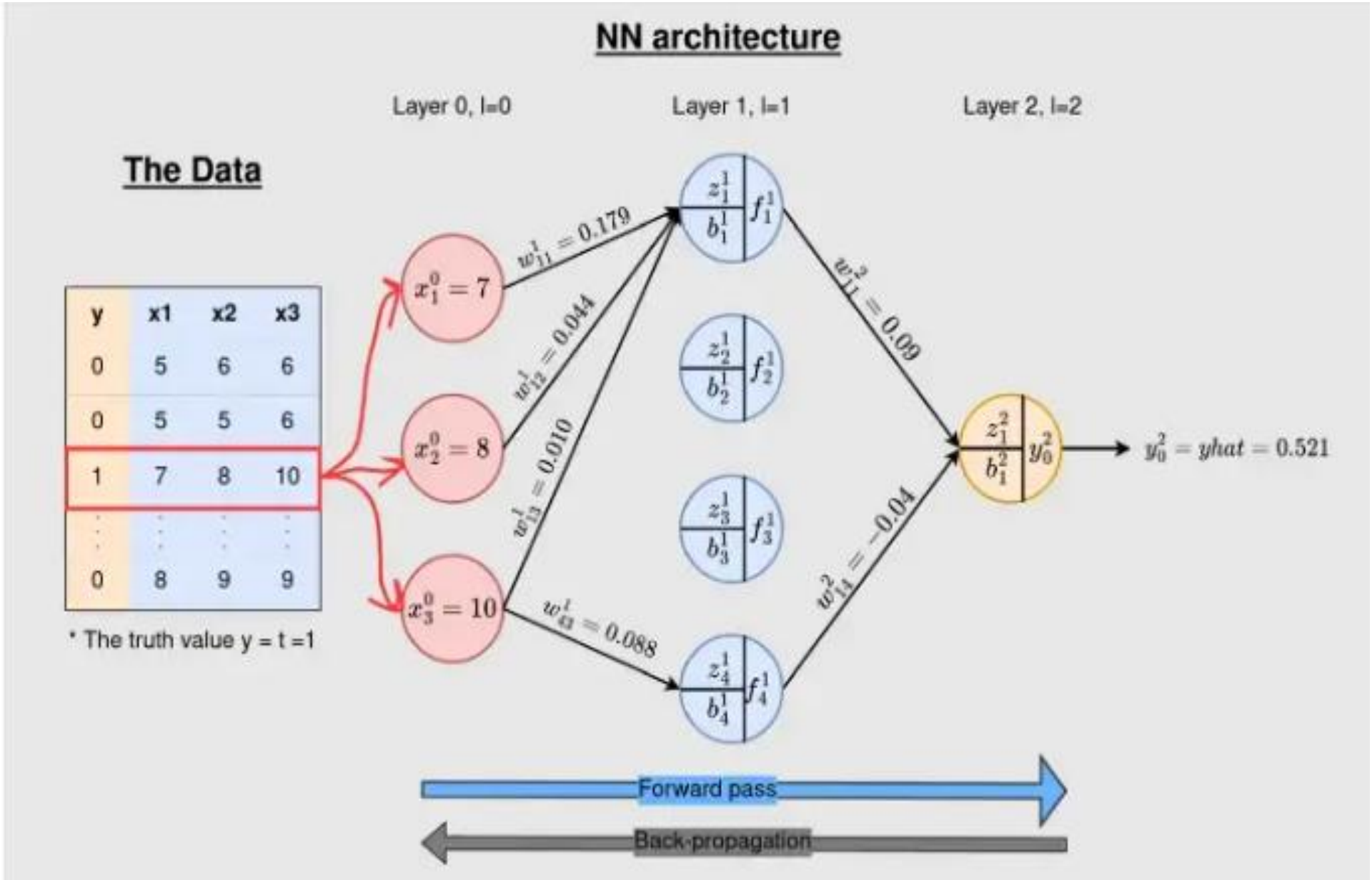
Summary of Results:

- Training (With Dropout and Batch Normalization): $\hat{y} \approx 0.620$
- Inference (With Batch Normalization, Without Dropout): $\hat{y} \approx 0.560$

Key Points:

1. **Batch normalization** helps stabilize the learning process by normalizing the activations of each layer.
2. **Dropout** further prevents overfitting by introducing randomness during training.
3. The combination of both techniques can lead to more robust and generalizable models, as seen in the improved output during training.

Back-Propagation Work in Neural Networks?



Back-Propagation Work in Neural Networks?

- Definition: Back-propagation is a method for supervised learning used by NN to update parameters to make the n etwork's predictions more accurate. The parameter opti mization process is achieved using an optimization algor ithm called gradient descent.

Back-Propagation Work in Neural Networks?

A forward pass yields a prediction (`yhat`) of the target (`y`) at a loss which is captured by a cost function (E) defined as:

$$E(\theta) = \frac{1}{m} \sum_{i=1}^{m} L(y, yhat)$$

where `m` is the number of training examples, and `L` is the error/loss incurred when the model predicts `yhat` instead of actual value `y` .

we consider back-propagation on `1` training example (`m=1`).
With this consideration, `E` , reduces to

$$E(\theta) = L(y, yhat)$$

Loss Function

- Cross Entropy Error:

$$
\begin{aligned}
E(\theta) &= -[t \ln(yhat) + (1-t) \ln(1-yhat)] \\
&= -[1 \ln(0.521) + (1-1) \ln(1-0.521)] \\
&= 0.652
\end{aligned}
$$

Initial Weights and Bias

Input layer (l=0)

	i	value
x_i^0	1	7
	2	8
	3	10

Hidden layer (l=1)

	i \ j	1	2	3	4
w_{ji}^1	1	0.179	-0.186	-0.008	-0.048
	2	0.044	-0.028	-0.063	-0.131
	3	0.01	-0.035	-0.004	0.088
b_j^1		0	0	0	0
f_j^1		0.845	0.132	0.354	0.377

Output layer (l=2)

	i	j=1
w_{ji}^2	1	0.088
	2	0.171
	3	0.005
	4	-0.04
b_j^2		0
yhat		0.521

- i - is the node in the previous layer, **l-1**,
- j in the node/unit in the current layer, **l**.
- Example, w_{ji}^1, means weight value connecting node i in layer **l-1** (**l=0** - the input) and node **j** in layer **1**.

* The output of a forward pass is yhat = **0.521**

* Remember that the truth value y=1.

Updating Parameters on the Output-Hidden Layer

Chain rule of differentiation

It states that: if y=f(u) and u=g(x) (that is y=f(g(x))) then derivative of y with respect to x is

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$$

Put it simply, work on derivative of the outside function before moving inside.

In our case, we have,

$$E = L(y, yhat),\ yhat = g(z) \text{ and } z = \mathbf{w}x + \mathbf{b}$$

Chain rule of differentiation

Updating Parameters on the Output-Hidden Layer

The parameters at each layer are updated with the following equations

$$w_{t+1} := w_t - \epsilon \frac{\partial E(\theta)}{\partial w}$$

$$b_{t+1} := b_t - \epsilon \frac{\partial E(\theta)}{\partial b}$$

Updating Parameters on the Output-Hidden Layer

the binary cross-entropy loss function defined as:

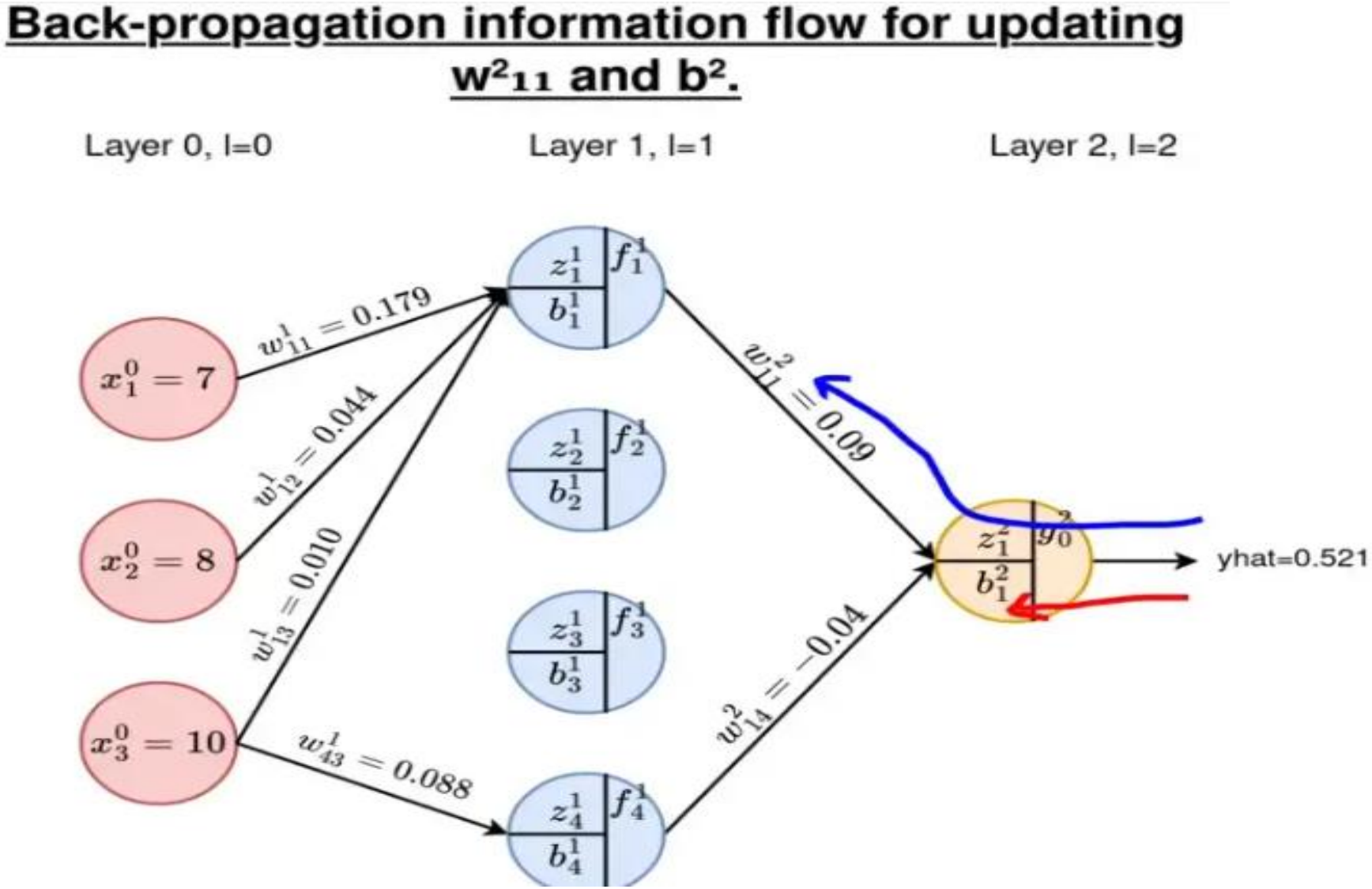
$$E(\theta) = -[t \ln(yhat) + (1 - t) \ln(1 - yhat)]$$

Equation 6: Binary cross entropy loss

We will use Sigmoid activation across all layers

$$g(z) = \frac{1}{1 - \exp^{-z}}$$

Updating Parameters on the Output-Hidden Layer



Derivative of Sigmoid Function

- The output $\hat{y}$ is the result of applying the **sigmoid activation function** to the pre-activation value z_o, where:

$$\hat{y} = \sigma(z_o) = \frac{1}{1 + e^{-z_o}}$$

  Here, $\sigma(z_o)$ is the sigmoid function.

Our goal is to calculate:

$$\frac{d\hat{y}}{dz_o}$$

This derivative tells us how much the output $\hat{y}$ changes when z_o (the pre-activation value) changes.

Derivative of Sigmoid Function

Step 1: Derivative of the Sigmoid Function

The sigmoid function is:

$$\sigma(z_o) = \frac{1}{1 + e^{-z_o}}$$

Now, we want to differentiate $\sigma(z_o)$ with respect to z_o. We use the **quotient rule** for differentiation because the sigmoid function is a ratio.

The quotient rule states that for a function $f(x) = \frac{g(x)}{h(x)}$, the derivative is:

$$f'(x) = \frac{g'(x)h(x) - g(x)h'(x)}{[h(x)]^2}$$

For the sigmoid function:

- $g(z_o) = 1$
- $h(z_o) = 1 + e^{-z_o}$

Let's differentiate:

1. $g(z_o) = 1$, so $g'(z_o) = 0$.
2. $h(z_o) = 1 + e^{-z_o}$, so $h'(z_o) = -e^{-z_o}$.

Derivative of Sigmoid Function

Now, applying the quotient rule:

$$\frac{d}{dz_o}\left(\frac{1}{1+e^{-z_o}}\right) = \frac{0 \cdot (1+e^{-z_o}) - 1 \cdot (-e^{-z_o})}{(1+e^{-z_o})^2}$$

Simplifying the numerator:

$$= \frac{e^{-z_o}}{(1+e^{-z_o})^2}$$

Derivative of Sigmoid Function

Step 2: Simplify the Derivative

Now, let's simplify the expression.

Recall that $\sigma(z_o) = \frac{1}{1+e^{-z_o}}$. Therefore, the simplified form of the derivative is:

$$\frac{d\hat{y}}{dz_o} = \sigma(z_o) \times (1 - \sigma(z_o))$$

This is a well-known result for the derivative of the sigmoid function.

Step 3: Substituting the Value of $\hat{y}$

Since $\hat{y} = \sigma(z_o)$, the derivative of $\hat{y}$ with respect to z_o becomes:

$$\frac{d\hat{y}}{dz_o} = \hat{y} \times (1 - \hat{y})$$

Updating Parameters on the Output-Hidden Layer

B1. Calculating Derivatives for the Weights

By chain rule of differentiation, we have

$$\frac{\partial E}{\partial w_{11}^2} = \frac{\partial E}{\partial yhat} \frac{\partial yhat}{z_1^2} \frac{\partial z_1^2}{\partial w_{11}^2}$$

and,

$$\begin{aligned} \frac{\partial z_1^2}{\partial w_{11}^2} &= \frac{\partial}{\partial w_{11}^2}(f_1^1 w_{11}^2 + f_2^1 w_{12}^2 + f_3^1 w_{13}^2 + f_4^1 w_{14}^2 + b_1^2) \\ &= f_1^1 \end{aligned}$$

$$\frac{\partial yhat}{\partial z_1^2} = \frac{\partial}{\partial z_1^2} g(z_1^2) = \frac{\partial}{\partial z_1^2}\left(\frac{1}{1+\exp^{-z_1^2}}\right) = (1 - yhat)yhat$$

$$\begin{aligned} \frac{\partial E}{\partial yhat} &= \frac{\partial}{\partial yhat}(-[t \ln yhat + (1-t)\ln(1-yhat)]) \\ &= -\frac{t}{yhat} + \frac{1-t}{1-yhat} \end{aligned}$$

Updating Parameters on the Output-Hidden Layer

$$
\begin{aligned}
\frac{\partial E}{\partial w_{11}^2} &= \frac{\partial E}{\partial yhat}\frac{\partial yhat}{z_1^2}\frac{\partial z_1^2}{\partial w_{11}^2} \\
&= -\frac{t}{yhat} + \frac{1-t}{1-yhat} \cdot (1-yhat)yhat \cdot f_1^1, \\
&= (yhat - t) \cdot f_1^1, \\
&= (0.521 - 1)0.845, \\
&= -0.405
\end{aligned}
$$

$$
\begin{aligned}
\frac{\partial E}{\partial w_{12}^2} &= -0.063, \\
\frac{\partial E}{\partial w_{13}^2} &= -0.169, \text{ and} \\
\frac{\partial E}{\partial w_{14}^2} &= -0.181.
\end{aligned}
$$

Updating Parameters on the Output-Hidden Layer

B2. Computing Derivatives for the bias

We need to compute

$$\frac{\partial E}{\partial b_1^2} = \frac{\partial E}{\partial yhat}\frac{\partial yhat}{\partial z^2}\frac{\partial z^2}{\partial b_1^2}$$

$$\frac{\partial z^2}{\partial b_1^2} = \frac{\partial}{\partial b_1^2}(f_1^1 w_{11}^2 + f_2^1 w_{12}^2 + f_3^1 w_{13}^2 + b_1^2) = 1$$

$$\begin{aligned}
\therefore \frac{\partial E}{\partial b_1^2} &= \frac{\partial E}{\partial yhat}\frac{\partial yhat}{\partial z^2}\frac{\partial z^2}{\partial b_1^2}, \\
&= -\frac{t}{yhat} + \frac{1-t}{1-yhat} \cdot (1-yhat)yhat \cdot 1, \\
&= (yhat - t) \cdot 1, \\
&= (0.521 - 1) \cdot 1 \\
&= -0.479
\end{aligned}$$

Updating Parameters on the Output-Hidden Layer

Gradients at the output-input

	i	j=1
$\frac{\partial E}{\partial w_{1i}^1}$	1	-0.405
	2	-0.063
	3	-0.169
	4	-0.181
$\frac{\partial E}{\partial b_1^2}$		-0.479

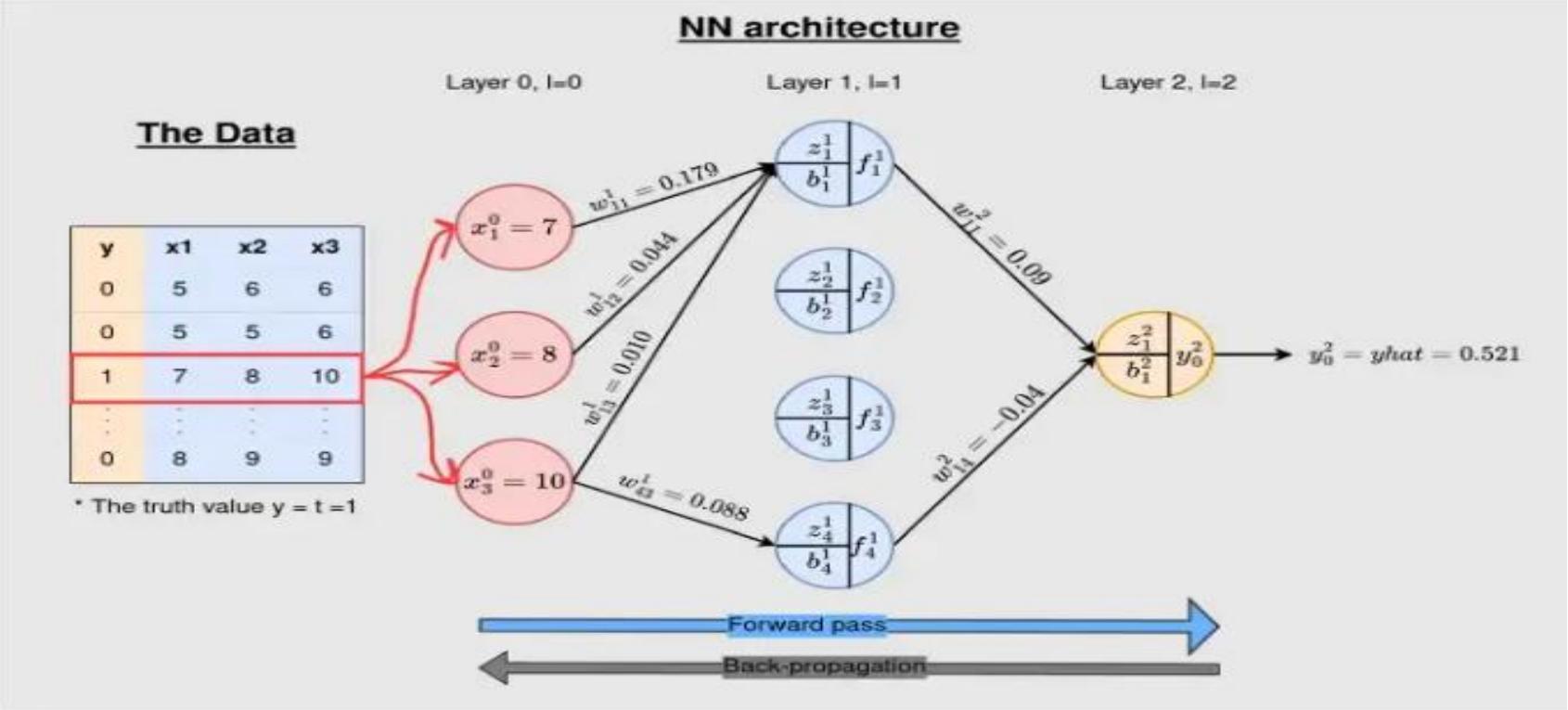
Updating Parameters on the Output-Hidden Layer

B3. Updating Parameters at the Output-Hidden Layers

$$
\begin{aligned}
w_{11}^2 &= w_{11}^2 - \epsilon \frac{\partial E}{\partial w_{11}^2}, \\
&= 0.088 - 0.5(-0.405), \\
&= 0.2905, \text{ and },
\end{aligned}
$$

$$
\begin{aligned}
b_1^1 &= b_1^2 - \epsilon \frac{\partial E}{\partial b_1^2}, \\
&= 0 - 0.5(-0.479), \\
&= 0.24.
\end{aligned}
$$

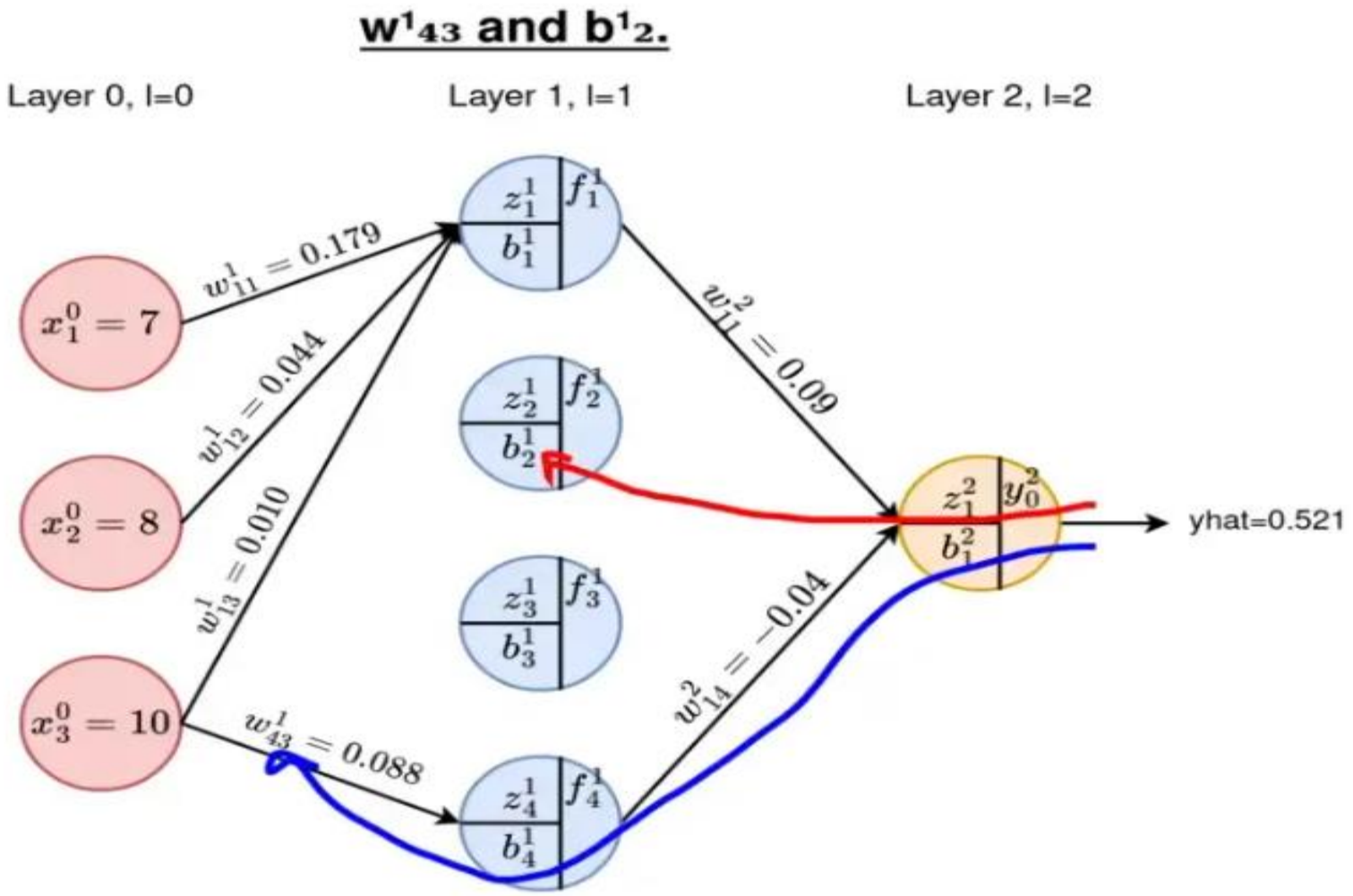
Updating Parameters on the Output-Hidden Layer



Updated parameters hidden-output

	i	old	new
w_{1i}^2	1	0.088	0.291
	2	0.171	0.203
	3	0.005	0.09
	4	-0.04	0.05
b_1^2		0	0.24

Updating Parameters at Hidden-Input Layer



Updating Parameters at Hidden-Input Layer

C1. Gradients of weights

$$\frac{\partial E}{\partial w_{43}^1} = \frac{\partial E}{\partial yhat} \frac{\partial yhat}{\partial z_1^2} \frac{\partial z_1^2}{\partial f_4^1} \frac{\partial f_4^1}{\partial z_4^1} \frac{\partial z_4^1}{\partial w_{43}^1}$$

- weighted inputs for layer `1`

$$\frac{\partial z_4^1}{\partial w_{43}^1} = \frac{\partial}{\partial w_{43}^1}(w_{41}^1 x_1^0 + w_{42}^1 x_2^0 + w_{43}^1 x_3^0 + b_4^1) = x_3^0$$

- derivative of Sigmoid activation function applied to the first layer

$$\frac{\partial f_4^1}{\partial z_4^1} = \frac{\partial}{z_4^1}\left(\frac{1}{1+\exp^{-f_4^1}}\right) = (1 - f_4^1) f_4^1$$

Updating Parameters at Hidden-Input Layer

- Weighted inputs to the output layer. `f` -values are the outputs of the hidden layer.

$$\frac{\partial z_1^2}{\partial f_4^1} = \frac{\partial}{\partial f_4^1}(f_1^1 w_{11}^2 + f_2^1 w_{12}^2 + f_3^1 w_{13}^2 + f_4^1 w_{14}^2 + b_1^2) = w_{14}^2$$

- Activation function applied to the output of the last layer.

$$\frac{\partial yhat}{\partial z_1^2} = \frac{\partial}{\partial z_1^2}\left(\frac{1}{1+\exp^{-yhat}}\right) = (1 - yhat)yhat$$

Updating Parameters at Hidden-Input Layer

- Derivative of binary cross-entropy loss function wrt to `yhat`.

$$\begin{aligned}\frac{E}{\partial yhat} &= \frac{\partial}{\partial yhat}(-t\ln(yhat) - (1-t)\ln(1-yhat)) \\ &= -\frac{t}{yhat} + \frac{1-t}{1-yhat}\end{aligned}$$

$$\begin{aligned}\frac{\partial E}{\partial w_{43}^1} &= \left(-\frac{t}{yhat} + \frac{1-t}{1-yhat}\right) \cdot (1-yhat)yhat \cdot w_{14}^2 \cdot (1-f_4^1)f_4^1 \cdot x_3^0, \\ &= (yhat - t) \cdot w_{14}^2 \cdot (1-f_4^1)f_4^1 \cdot x_3^0, \\ &= (0.521-1)(-0.04) \cdot (1-0.377)0.377 \cdot 10, \\ &= 0.045.\end{aligned}$$

Updating Parameters at Hidden-Input Layer

C2. Gradients of bias

Using the same concepts as before, check that, for b^1_2, we have

$$\begin{aligned}\frac{\partial E}{\partial b_3^1} &= \frac{\partial E}{\partial yhat}\frac{\partial yhat}{\partial z_1^2}\frac{\partial z_1^2}{\partial f_3^1}\frac{\partial f_3^1}{\partial z_3^1}\frac{\partial z_3^1}{\partial b_3^1} \\ &= (yhat - t)\cdot w_{13}^2 \cdot (1 - f_3^1) f_3^1 \cdot 1 \\ &= (0.521 - 1)\cdot (0.005)\cdot (1 - 0.354)0.354 \cdot 1 \\ &= -0.000547698\end{aligned}$$

Updating Parameters at Hidden-Input Layer

All gradient values for the hidden-input are tabulated below

The gradients of hidden-input layers

	j / i	1	2	3	4
$\frac{\partial E}{\partial w_{ji}^1}$	1	-0.039	-0.066	-0.004	0.032
	2	-0.044	-0.075	-0.004	0.036
	3	-0.055	-0.094	-0.005	0.046
$\frac{\partial E}{\partial b_j^1}$		-0.006	-0.009	-0.001	0.005

Updating Parameters at Hidden-Input Layer

C3. Updating Parameters at the Hidden-Input

Lets go back to the update equations and work on updating w^1_{13} and b^1_3

$$
\begin{aligned}
w^1_{13} &= w^1_{13} - \epsilon \frac{\partial E}{\partial w^1_{13}} \\
&= 0.01 - 0.5(-0.055) \\
&= 0.0375
\end{aligned}
$$

$$
\begin{aligned}
b^1_3 &= b^1_3 - \epsilon \frac{\partial E}{\partial b^1_3} \\
&= 0 - 0.5(-0.001) \\
&= 0.001
\end{aligned}
$$

Updating Parameters at Hidden-Input Layer

Updated parameters for hidden-input layers

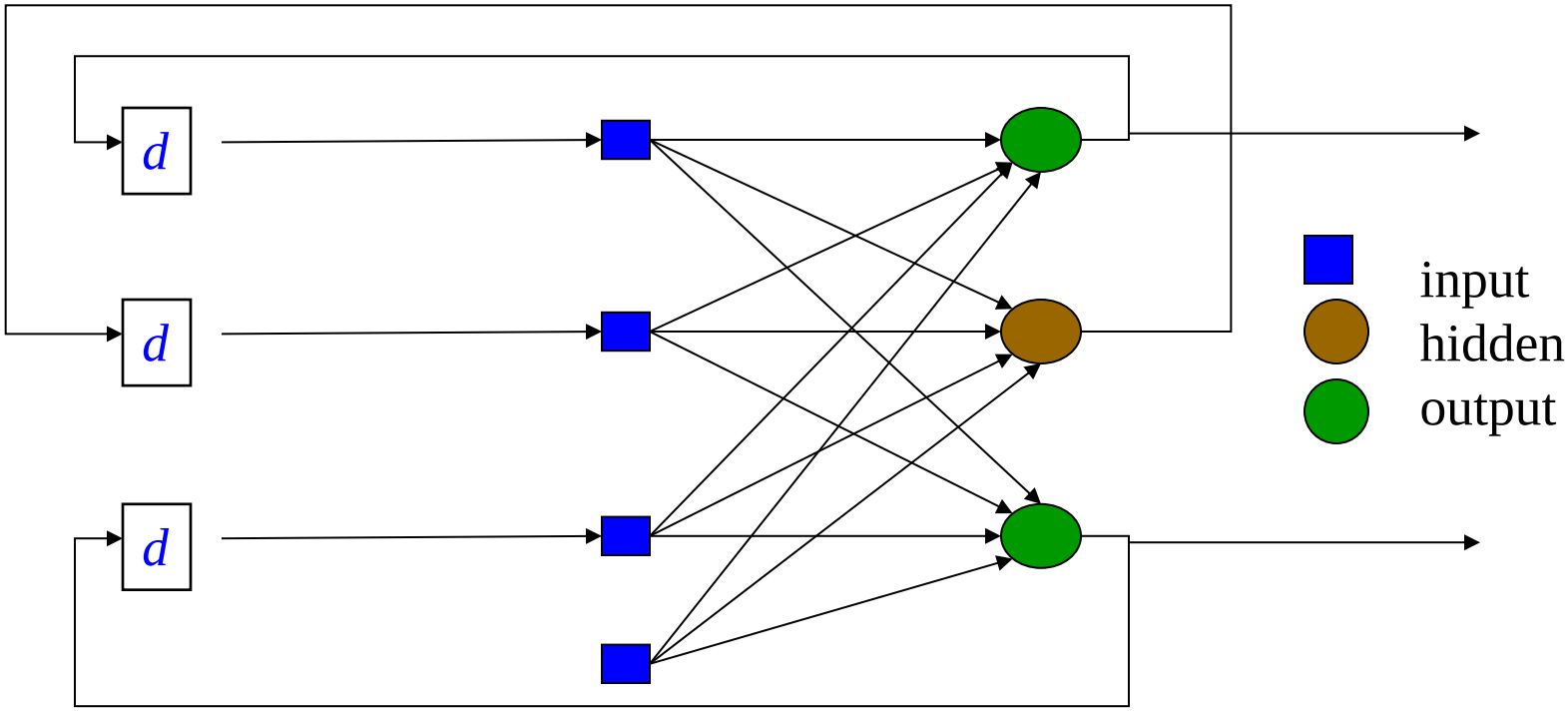
	i \ j	1	2	3	4
w_{ji}^1	1	0.198	-0.153	-0.006	-0.064
	2	0.066	0.01	-0.061	-0.150
	3	0.037	0.012	-0.002	0.066
b_j^1		0.003	0.005	0	-0.002

Recurrent Network

- **FFNN** is acyclic where data passes from input to the output nodes and not vice versa.
  - Once the FFNN is trained, its state is fixed and does not alter as new data is presented to it. It does not have memory.
- **Recurrent network** can have connections that go backward from output to input nodes and models dynamic systems.
  - In this way, a recurrent network's internal state can be altered as sets of input data are presented. It can be said to have memory.
  - It is useful in solving problems where the solution depends not just on the current inputs but on all previous inputs.
- Applications
  - predict stock market price,
  - weather forecast

Recurrent Network Architecture

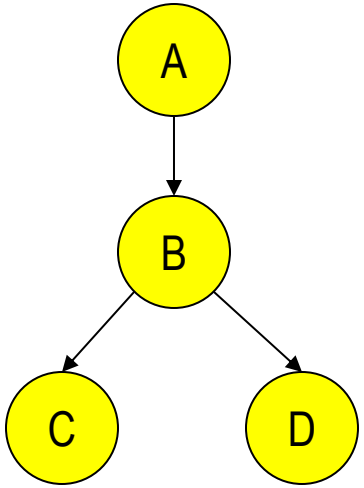
- Recurrent Network with ***hidden neuron***: unit delay operator *d* is used to model a dynamic system



A Bayesian Belief Network

A Bayesian belief network is made up of:

1. A Directed Acyclic Graph



2. A set of conditional probability tables for each node in the graph

A	P(A)
false	0.6
true	0.4

A	B	P(B|A)
false	false	0.01
false	true	0.99
true	false	0.7
true	true	0.3

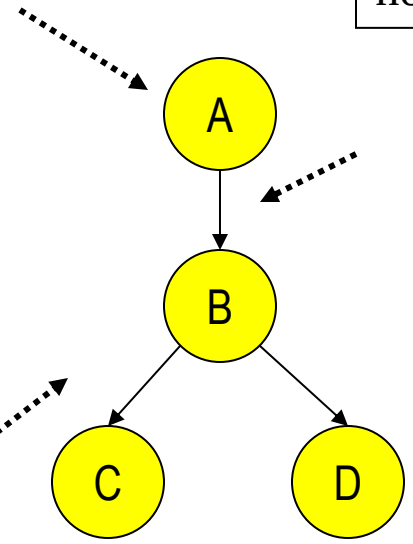
B	D	P(D|B)
false	false	0.02
false	true	0.98
true	false	0.05
true	true	0.95

B	C	P(C|B)
false	false	0.4
false	true	0.6
true	false	0.9
true	true	0.1

A Directed Acyclic Graph

Each node in the graph is a random variable

A node ***X*** is a parent of another node *Y* if there is an arrow from node ***X*** to node ***Y*** e.g. *A* is a parent of *B*



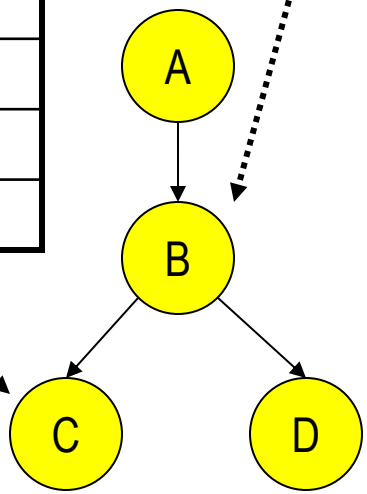
Informally, an arrow from node ***X*** to node ***Y*** means ***X*** has a direct influence on ***Y***

A Set of Tables for Each Node

A	P(A)
false	0.6
true	0.4

A	B	P(B|A)
false	false	0.01
false	true	0.99
true	false	0.7
true	true	0.3

B	C	P(C|B)
false	false	0.4
false	true	0.6
true	false	0.9
true	true	0.1



B	D	P(D|B)
false	false	0.02
false	true	0.98
true	false	0.05
true	true	0.95

Each node X_i has a conditional probability distribution P(X_i | Parents(X_i)) that quantifies the effect of the parents on the node

The parameters are the probabilities in these conditional probability tables (CPTs)

A Set of Tables for Each Node (Cont…..)

Conditional Probability Distribution for C given B

B	C	P(C|B)
false	false	0.4
false	true	0.6
true	false	0.9
true	true	0.1

For a given combination of values of the parents (B in this example), the entries for P(C=true | B) and P(C=false | B) must add up to 1
eg. P(C=true | B=false) + P(C=false |B=false)=1

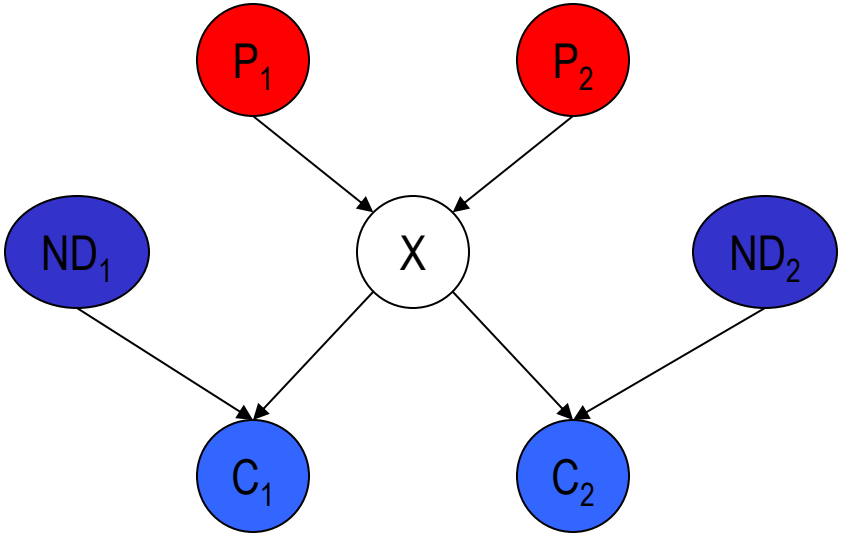
Properties

Two important properties:

- Encodes the conditional independence relationships between the variables in the graph structure
- Is a compact representation of the joint probability distribution over the variables

Conditional Independence

The Markov condition: given its parents (P_1, P_2),
a node (X) is conditionally independent of its non-des
cendants (ND_1, ND_2)



The Joint Probability Distribution

Due to the Markov condition, we can compute the joint probability distribution over all the variables $X_1, \ldots, X_n$ in the Bayesian net using the formula:

$$P(X_1 = x_1, \ldots, X_n = x_n) = \prod_{i=1}^{n} P(X_i = x_i \mid Parents(X_i))$$

Where Parents(X_i) means the values of the Parents of the node X_i with respect to the graph

Using a Bayesian Network Example

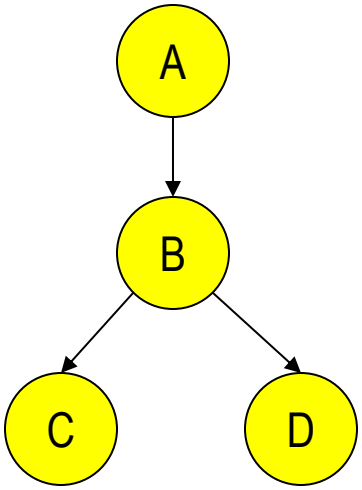
Using the network in the example, suppose we want to calculate:

P(A = true, B = true, C = true, D = true)

= P(A = true) * P(B = true | A = true) *

P(C = true | B = true) P(D = true | B = true)

= (0.4)*(0.3)*(0.1)*(0.95)



Using a Bayesian Network Example

Using the network in the example, suppose you want to calcul ate:

P(A = true, B = true, C = true, D = true)

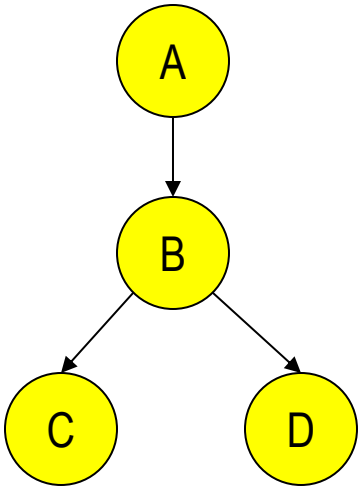
= P(A = true) * P(B = true | A = true) *

P(C = true | B = true) P(D = true | B = true)

= (0.4)*(0.3)*(0.1)*(0.95)

This is from the graph structure

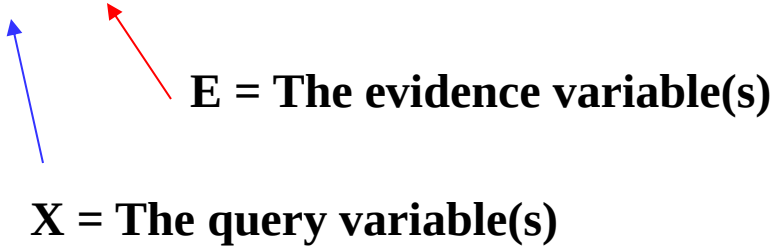
These numbers are from the conditional probability tables



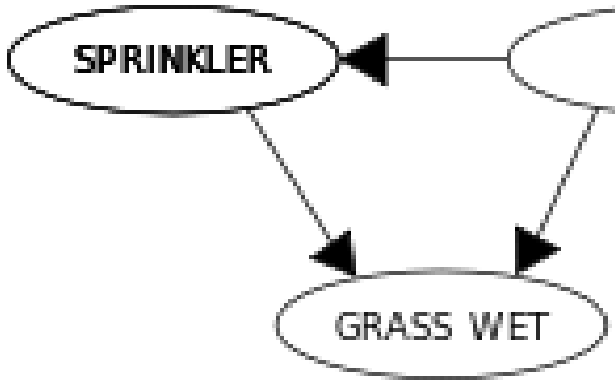
Inference

- Using a Bayesian network to compute probabilities is called inference
- In general, inference involves queries of the form:

  $P(X \mid E)$



Bayesian belief network example



RAIN	SPRINKLER = T	SPRINKLER = F
F	0.4	0.6
T	0.01	0.99

RAIN = T	RAIN = F
0.2	0.8

SPRINKLER	RAIN	GRASS WET = T	GRASS WET = F
F	F	0.0	1.0
F	T	0.8	0.2
T	F	0.9	0.1
T	T	0.99	0.01

Figure . A simple Bayesian network with conditional probability tables

Bayesian belief network example

- Suppose that there are two events which could cause grass to be wet: either the sprinkler is on or it's raining. Also, suppose that the rain has a direct effect on the use of the sprinkler (namely that when it rains, the sprinkler is usually not turned on). Then the situation can be modeled with a Bayesian network (shown). All three variables have two possible values, T (for true) and F (for false).
- The joint probability function is:

$$P(G,S,R) = P(G|S,R)\,P(S|R)\,P(R)$$

where the names of the variables have been abbreviated to G = Grass wet (yes/no), S = Sprinkler turned on (yes/no), and R = Raining (yes/no).

Bayesian belief network example

- The model can answer questions like "***What is the probability that it is raining, given the grass is wet***?"

- By using the conditional probability formula and summing over all random variables:

$$\mathrm{P}(R = T \mid G = T) = \frac{\mathrm{P}(G = T, R = T)}{\mathrm{P}(G = T)} = \frac{\sum_{S \in \{T,F\}} \mathrm{P}(G = T, S, R = T)}{\sum_{S,R \in \{T,F\}} \mathrm{P}(G = T, S, R)}$$

Bayesian belief network example

- Using the expansion for the joint probability function P (G,S,R) and the conditional probabilities from the conditional probability tables (CPTs) stated in the diagram, we can evaluate each term in the sums in the numerator and denominator. For example,

$$
\begin{aligned}
&\mathrm{P}(G = T, S = T, R = T) \\
&\quad = \mathrm{P}(G = T \mid S = T, R = T)\mathrm{P}(S = T \mid R = T)\mathrm{P}(R = T) \\
&\quad = 0.99 \times 0.01 \times 0.2 \\
&\quad = 0.00198.
\end{aligned}
$$

Bayesian belief network example

- Then the numerical results (subscripted by the associated variable values) are :

$$\mathrm{P}(R = T \mid G = T) = \frac{\mathrm{P}(G = T, R = T)}{\mathrm{P}(G = T)} = \frac{\sum_{S \in \{T,F\}} \mathrm{P}(G = T, S, R = T)}{\sum_{S,R \in \{T,F\}} \mathrm{P}(G = T, S, R)}$$

$$= \frac{(0.99 \times 0.01 \times 0.2 = 0.00198_{TTT}) + (0.8 \times 0.99 \times 0.2 = 0.1584_{TFT})}{0.00198_{TTT} + 0.288_{TTF} + 0.1584_{TFT} + 0_{TFF}} \approx 35.77\%.$$

Thank You